AULA 3

# Aumentando seu repertório

## Relembrando ...

Até agora construímos a "cara" do nosso sistema.

Ele tem uma interface bonita, tem conexão com LLMs, e tem *streaming* de respostas. Parece completo, não? Mas falta algo essencial.

E pra isso... começamos uma nova fase: dar memória e contexto ao sistema.

Pense comigo: de que adianta um assistente inteligente se ele não lembra de informações importantes? Se a cada conversa ele começa do zero? Se ele não consegue consultar dados históricos da sua empresa? É chato ter que explicar tudo de novo a cada conversa, concorda?

Por isso, vamos adicionar uma solução de banco de dados analítico ao nosso projeto. Com isso, nossa IA poderá responder perguntas baseadas em dados reais, sem "inventar" respostas como faz uma LLM pura (lembra do problema das alucinações que discutimos na semana passada?).

**POR EXEMPLO...**

Para que isso fique mais claro, pense em uma concessionária de carros.

Uma LLM como o ChatGPT consegue responder várias perguntas sobre veículos, como preços de tabela, especificações técnicas, e fazer comparativos entre modelos porque estas informações estão na internet. Mas e se tivermos perguntas como:

- Qual carro vocês têm em estoque com câmbio automático?
- Qual foi o desconto médio dado no último mês?
- Quantos carros vendemos na semana passada?

Nenhuma LLM consegue responder isso hoje sem acesso aos dados internos da concessionária. Esses dados estão em sistemas internos, planilhas e bancos de dados.

E é exatamente isso que vamos aprender a conectar.

**#FICADICA**

Isso é algo que frequentemente perguntam em entrevistas para vagas de data science e ML engineering.

Recrutadores querem saber se você entende a diferença entre o que uma LLM "sabe" (conhecimento geral) e o que ela precisa "consultar" (dados específicos da empresa). Se você souber explicar essa distinção com clareza, já sai na frente de outros candidatos.

Vamos começar ⬇️

## OLTP X OLAP

Antes de avançarmos, preciso explicar uma distinção fundamental no mundo de bancos de dados: OLTP e OLAP.

A grande maioria dos cursos de TI se preocupa com bancos do tipo OLTP, enquanto focaremos OLAP. Entender essa diferença é importante para quem quer trabalhar com dados e IA.

Vou usar uma analogia para entendermos isto melhor.

**POR EXEMPLO...**

Pense um supermercado movimentado. Existem dois tipos de operações acontecendo simultaneamente:

1. **No caixa (OLTP):** a cada segundo, dezenas de produtos são "bipados". Cada "bipe" é uma transação: o sistema registra o produto, atualiza o estoque, calcula o preço, aplica descontos.

Perceba que podemos ter milhares de pequenas operações acontecendo ao mesmo tempo, e todas precisam ser rápidas e precisas. Se o sistema travar por 10 segundos, forma-se fila, clientes reclamam, e o supermercado perde dinheiro.

2. **Na sala da diretoria (OLAP):** o gerente regional quer saber "qual foi a categoria de produtos que mais vendeu no trimestre passado em todos os 50 supermercados da rede?".

Essa pergunta não precisa de resposta em milissegundos, e pode levar alguns segundos (ou minutos). Mas ela precisa analisar milhões de registros de vendas de todas as lojas, agregar por categoria, comparar períodos. É uma operação completamente diferente. E, ainda, é uma operação pesada que não deve interferir com o pessoal trabalhando no caixa.

Percebe como são dois mundos? O caixa precisa de velocidade em operações pequenas. A diretoria precisa de capacidade de análise em grandes volumes. E é por isso que existem dois tipos de bancos de dados.

Vejamos cada um deles ...

## *Online transaction processing* (OLTP)

OLTP é o tipo de banco que é ensinado em disciplinas de banco de dados na maioria dos cursos de TI das universidades, e é o que a maioria dos profissionais de TI conhece.

Exemplos clássicos que as empresas usam são: SQL Server, PostgreSQL e MySQL.

Esse tipo de banco de dados serve para armazenar transações (ou seja, operações do dia a dia).

Alguns exemplos de transações são:

- Um pedido que alguém faz em um *e-commerce*.
- A atualização de um cadastro de cliente.
- Um saque em um caixa eletrônico.
- Uma transferência PIX.

Essas transações compartilham algumas necessidades em comum. Portanto, o foco do OLTP é em:

1

### Consistência

Se você transferiu R$ 100, o dinheiro precisa sair de uma conta e entrar na outra. Não pode "sumir" no meio do caminho.

2

### Velocidade

Cada operação precisa ser rápida (milissegundos).

3

### Concorrência

Muitas pequenas operações acontecendo ao mesmo tempo, sem conflito.

## *Online analytical processing* (OLAP)

Agora, várias empresas que trabalham com ciência de dados, análise de dados e IA generativa também possuem um segundo tipo de banco de dados.

Esse segundo tipo não tem como foco armazenar e processar transações, mas, sim, processar análises.

Deixe-me contar um exemplo real.

Percebe a diferença?

O OLTP registra cada "bipe" individual. O OLAP junta milhões de "bipes" para responder perguntas de negócio.

No Nubank, onde trabalhei depois, a lógica era a mesma.

Os sistemas transacionais (OLTP) processavam cada PIX, cada pagamento de fatura, cada compra no cartão.

Já os sistemas analíticos (OLAP) agregavam tudo para responder perguntas como "Qual é a taxa de inadimplência por faixa etária?" ou "Qual canal de aquisição traz clientes mais rentáveis?".

**Toda empresa de tecnologia madura tem os dois.**

## Comparativo: OLTP x OLAP

Vamos consolidar as diferenças em uma tabela:

| Característica | OLTP | OLAP |
|---|---|---|
| **Operações típicas** | INSERT, UPDATE, DELETE | SELECT com agregações |
| **Volume por operação** | Poucos registros | Milhões de registros |
| **Velocidade** | Transações rápidas | *Queries* complexas rápidas |
| **Exemplos** | PostgreSQL, MySQL | DuckDB, ClickHouse, BigQuery |
| **Caso de uso** | Aplicações do dia a dia | *Dashboards*, análises |

O autor.

## OLAP para IA

Agora, precisamos usar OLAP para o nosso projeto de IA por três motivos básicos.

- O **primeiro** é a *performance* em agregações. Uma pessoa que poderia usar a nossa IA perguntaria algo como "Qual foi o total de vendas do mês passado?", e para isso ela precisa somar milhões de registros. Bancos OLAP são otimizados para esse tipo de operação.
- O **segundo** motivo está na imprevisibilidade do tipo de pergunta que os nossos usuários poderiam fazer na IA. Bancos OLAP permitem *queries* flexíveis sobre grandes volumes sem precisar criar índices específicos para cada tipo de pergunta.

- Já o **terceiro** motivo é a integração: é fácil integrarmos um banco OLAP com o Python, pandas, ferramentas de BI, e com *pipelines* de IA.

Agora, não sei se percebeu, mas nos exemplos de OLAP da tabela acima eu listei o DuckDB. Ele é um banco de dados OLAP excelente para prototipagem rápida, que é exatamente o que estamos fazendo aqui.

Veja algumas características que o tornam ideal:

| Característica | Benefício para nós |
|---|---|
| *In-process* | Roda dentro do Python, sem precisar instalar servidor. |
| **Fácil de configurar** | Não precisamos nos esforçar para que ele funcione corretamente. |
| **Leitura direta de arquivos** | Faz *query* direto em CSV, Parquet, JSON sem importar. |
| ***Performance* excepcional** | Até 100x mais rápido que pandas em algumas operações. |
| **SQL completo** | Suporta JOINs, *window functions*, agregações de dados, entre outros. |
| **Integração com pandas** | Converte resultados para *dataframe* com um comando. |

O autor.

**FIQUE ATENTO**

O DuckDB é usado por algumas das maiores empresas do mundo. Aprender a usá-lo agora é um diferencial competitivo.

Uma analogia que gosto de usar ⬇️

O DuckDB é como o SQLite do mundo analítico. Assim como o SQLite é um banco OLTP que roda "embutido" (sem servidor), o DuckDB é um banco OLAP que roda embutido. Isso significa: sem Docker, sem configuração de rede, sem dor de cabeça. Perfeito para protótipos e projetos de IA.

## *Dataframes* e Pandas

Antes de integrarmos o DuckDB ao nosso projeto, preciso garantir que estamos na mesma página sobre alguns conceitos.

Você provavelmente já ouviu falar de *dataframes* e pandas, mas vamos revisar rapidamente para que tudo fique claro.

### O que é um dataframe?

Pense em uma planilha do Excel. Você tem linhas, colunas, e cada célula contém um valor. Um *dataframe* é exatamente isso, só que dentro do Python.

Analise o exemplo a seguir:

| Nome | Idade | Cidade | Salário |
|---|---|---|---|
| Ana | 28 | São Paulo | 5500 |
| Bruno | 35 | Curitiba | 7200 |
| Carla | 42 | Rio de Janeiro | 9800 |
| Daniel | 31 | Belo Horizonte | 6100 |

O autor.

Essa estrutura tabular (linhas representando registros e colunas representando atributos) é um *dataframe*.

A diferença é que, em vez de clicar em células com o *mouse*, você manipula os dados com código Python. E isso abre um mundo de possibilidades: você pode filtrar milhões de linhas em segundos, calcular estatísticas, cruzar tabelas, e muito mais.

## E o que é o pandas?

Pandas é a biblioteca mais popular do Python para manipulação de dados tabulares. Quando alguém fala "vou analisar esses dados em Python", em 90% dos casos está usando pandas.

Com ele, podemos:

- Carregar dados de arquivos CSV, Excel, JSON, bancos de dados.
- Filtrar registros com condições (ex.: "apenas vendas acima de R$ 1.000").
- Agrupar e agregar (ex.: "total de vendas por região").
- Transformar colunas (ex.: "converter datas para mês/ano").
- Exportar para diversos formatos.

## Integrando DuckDB e pandas

Aqui está o ponto que torna o DuckDB especial para cientistas de dados: ele tem uma ótima integração com pandas. Isso significa que podemos tanto usar o DuckDB para fazermos consultas diretamente em um *dataframe* do pandas, como também usar o pandas para converter dados do DuckDB em *dataframes*.

**#FICADICA**

Para arquivos grandes (milhões de linhas), carregar com DuckDB é **muito mais rápido** que usar o pandas. Em alguns benchmarks, a diferença chega a 10-50x. Isso porque o DuckDB foi projetado para processar grandes volumes, enquanto o Pandas foi projetado para flexibilidade.

## E quando usar cada um?

Uma dúvida comum é: "se o DuckDB é tão rápido, por que não usar só ele?".

Boa pergunta! Na prática, cada ferramenta tem seu ponto forte, observe:

| Situação | Melhor escolha |
|---|---|
| Carregar arquivo grande (milhões de linhas). | DuckDB |
| Agregações e JOINs complexos. | DuckDB (SQL) |
| Manipulações linha a linha. | Pandas |
| Visualização rápida (gráficos). | Pandas + matplotlib/seaborn |
| Integração com bibliotecas de ML. | Pandas (scikit-learn espera DataFrames) |
| Prototipagem rápida de *queries*. | DuckDB (SQL é mais legível para agregações) |

O autor.

**#FICADICA**

Na minha experiência profissional, o padrão que funciona bem é:

DuckDB para carregar e agregar.

Pandas para refinar e visualizar.

Você usa o poder do DuckDB para processar os dados pesados, converte para dataframe, e aí usa pandas para os ajustes finais e gráficos.

Mas vale atenção aqui ⬇️

Veja que essa discussão que acabamos de ter é uma decisão técnica de arquitetura. Esse é o tipo de decisão que vibecoding (ainda) não resolve bem. É por isso que também estudamos esse tipo de assunto no curso.

## Construindo um mini *data warehouse*

Agora que você entende a integração DuckDB-Pandas, vamos ao nosso objetivo: construir um pequeno *data warehouse* (DW) para o projeto.

Um DW é um banco de dados OLAP (ou seja, um banco otimizado para **análises**, não para transações).

Vamos usar uma analogia.

Imagine que você acabou de comprar uma rede de pizzarias com 20 lojas. E você descobriu que cada loja tem seu próprio sisteminha:

- Loja 1 usa um *software* de caixa antigo.
- Loja 2 usa uma planilha Excel.
- Loja 3 usa um *app* moderno.
- E assim por diante...

Cada sistema registra as vendas do dia, mas de formas diferentes.

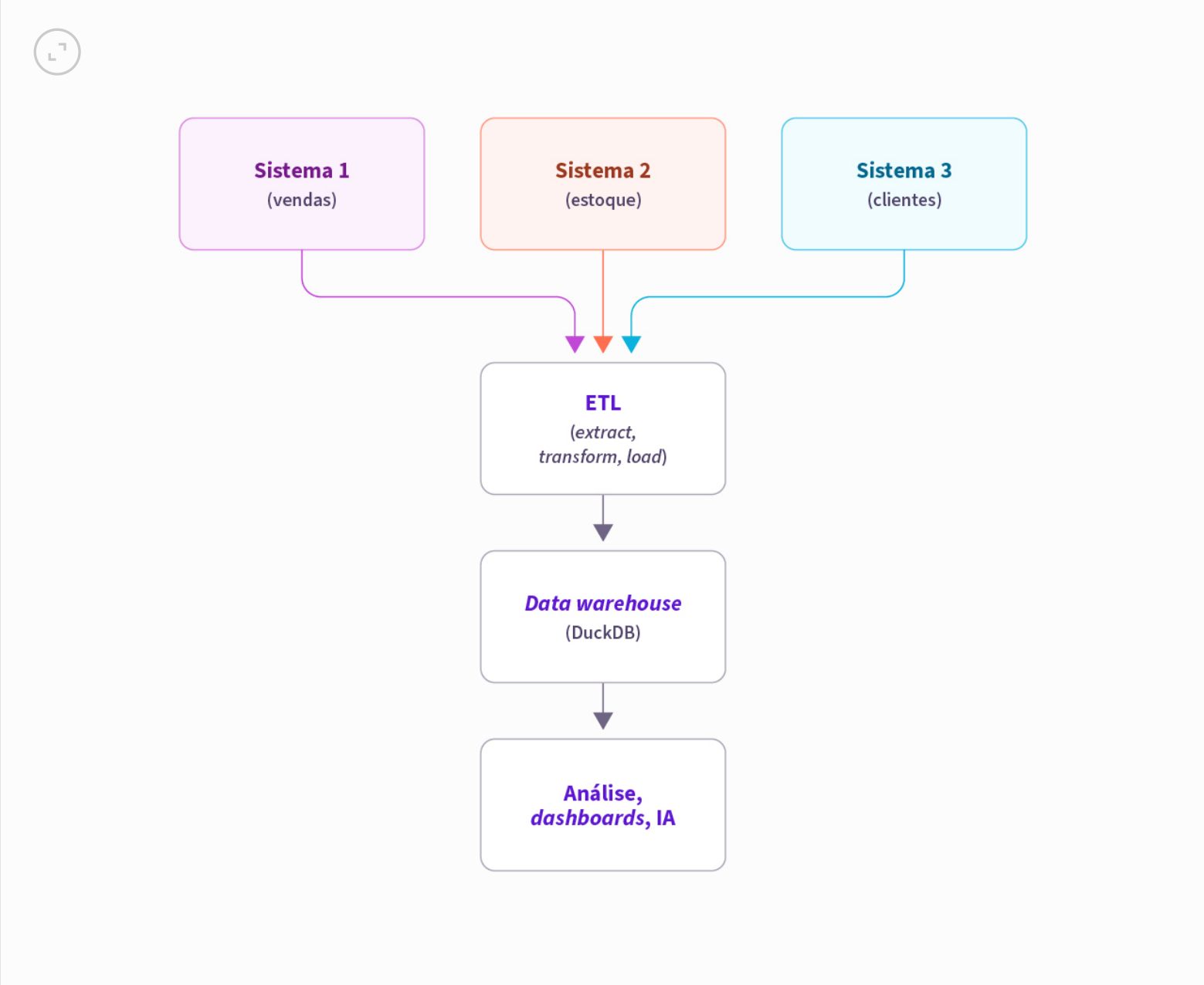
Agora, você quer responder: "Qual sabor de pizza mais vendeu em toda a rede no último mês?".

Para responder isso, você teria que entrar no sistema de cada loja, exportar os dados individualmente, padronizar os dados, juntar tudo em um lugar, e aí fazer a análise.

A ideia do DW é centralizar isso. Dessa forma, ele extrai dados de várias fontes (os sistemas das lojas), transforma para um formato padronizado (mesmas colunas, mesmos tipos), e carrega em um banco otimizado para consultas (DuckDB, no nosso caso). Esse processo é o famoso **ETL** que mencionei antes.

Algo assim:

#ParaTodosVerem




O autor.

## E por que um DW no nosso projeto de IA?

Se a LLM (GPT, Claude) tem conhecimento geral do mundo, o DW tem conhecimento específico do negócio. Com isso, a IA será capaz de entender a pergunta (usando LLM), traduzir para uma query SQL, executar no *data warehouse* (DuckDB), e formatar a resposta em linguagem natural. O nome disso é Text-to-SQL ou Natural Language to SQL, e é uma das aplicações mais práticas de IA em empresas hoje.

O que acha de usarmos ele no nosso projeto? Na videoaula a seguir, vamos colocar a mão na massa e criar nosso primeiro mini DW.

Para visualizar o vídeo, aponte a câmera do seu celular

© PUCPR - Todos os direitos reservados